

Lab Assignment Report: Error Detection Module

Sajjad Ahmed Shaaz
002410501093
BCSE-III A3

August 17, 2026

Basic Information

- **Subject:** CSE/PC/B/S/314 Computer Networks Lab
- **Assignment Number:** 1
- **Problem Statement:** Design and implement an error detection module utilizing Checksum (16-bit) and Cyclic Redundancy Check (CRC) schemes over a simulated network environment.

1 Design

Purpose of the Program. The primary objective of this program is to simulate network data transmission and implement robust error detection mechanisms. By framing data, injecting various types of errors, and transmitting codewords between a sender and receiver, the program evaluates the efficiency and accuracy of 16-bit Checksum and CRC algorithms (CRC-8, CRC-10, CRC-16, and CRC-32).

Design Rationale. The two schemes are deliberately implemented behind a single `calculateFCS` interface so that the sender, receiver, and offline driver all share one code path and cannot disagree on how a frame check sequence is produced. The checksum and CRC differ only in the function selected by the `FCS` enum; framing, transmission, and verification are identical across schemes. Verification is done by *recompute-and-compare* rather than by checking for a zero remainder: the receiver recomputes the FCS over the received dataword and compares it to the received FCS. This is equivalent to the textbook zero-remainder / zero-checksum property but keeps a single implementation for all five schemes and avoids a separate “syndrome” code path. The transport is UDP because it is message-oriented — one datagram maps cleanly to one frame — which keeps the simulated link close to the frame abstraction being tested.

Frame Format. Each frame follows a fixed layout modelled on the Ethernet header, summarized in Table 1. The dataword (over which the FCS is computed) is the concatenation of every field except the FCS itself; short payloads are zero-padded to the 46-byte minimum before the FCS is calculated.

Field	Size	Purpose
Source MAC	6 bytes	Sender hardware address
Destination MAC	6 bytes	Receiver hardware address
Payload length	2 bytes	Original (unpadded) payload length
Sequence number	2 bytes	Per-frame ordering index
Payload	46 bytes	Data chunk (zero-padded if short)
FCS	1–4 bytes	Checksum-16 / CRC remainder

Table 1: Frame layout. The FCS width depends on the scheme: 2 bytes for Checksum-16 and CRC-16, 1 byte for CRC-8, 2 bytes for CRC-10 (10 bits, zero-padded), and 4 bytes for CRC-32.

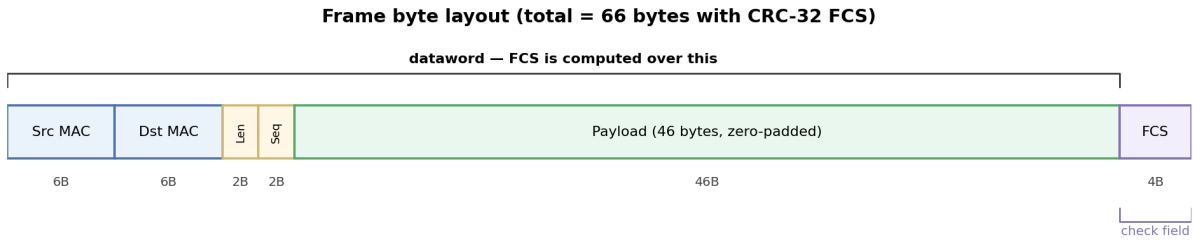


Figure 1: Frame byte layout drawn to scale. The dataword (all fields except the FCS) is what the check sequence is computed over; the payload is zero-padded to 46 bytes before computation.

Structure Diagram (Procedural Organization). The overall procedural organization is shown in Figure 2. The sender frames the input, computes the FCS, optionally injects an error, and transmits over UDP; the receiver parses the datagram, recomputes the FCS, and accepts or rejects the frame. A shared module (`common.cpp`) provides the checksum and CRC routines to both endpoints, and an offline driver runs automated trials.

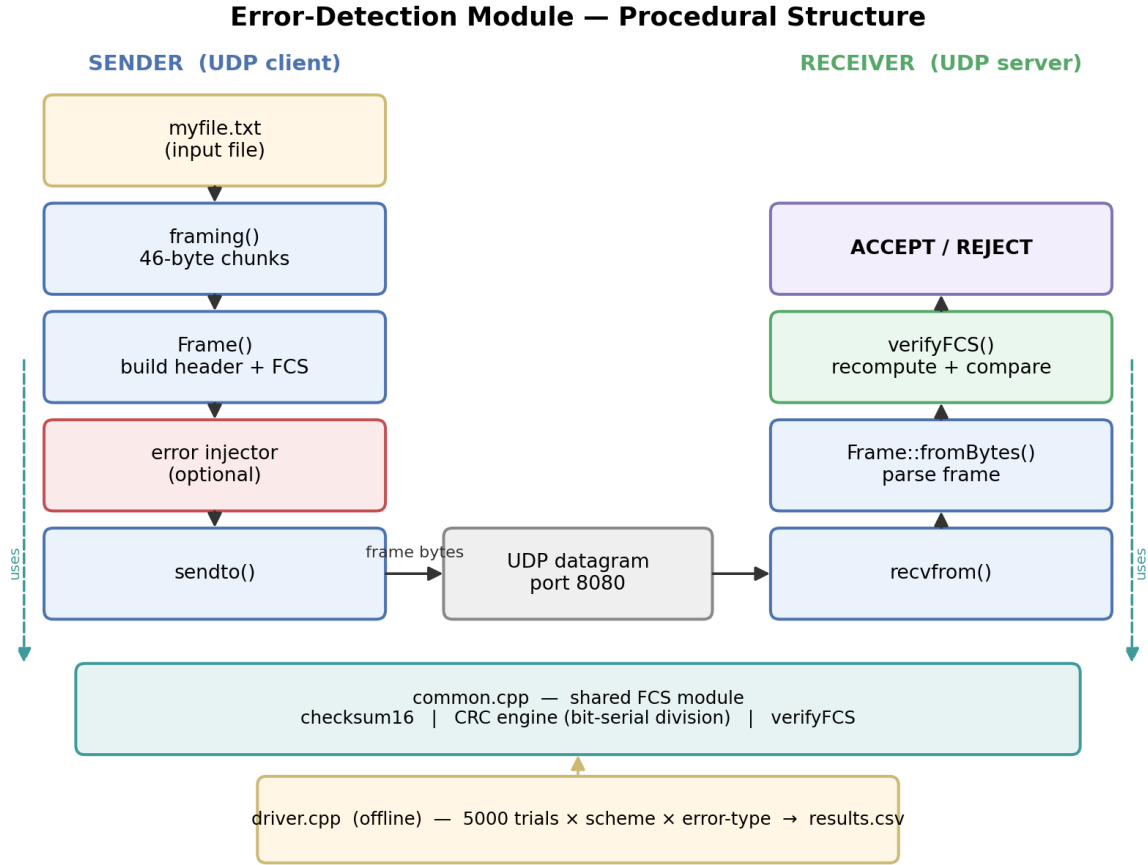


Figure 2: Procedural structure of the error-detection module.

The logical pipeline is:

1. **Input Reading:** Read dataword payload from the input file.
2. **Framing:** Split the byte stream into fixed 46-byte payloads (the Ethernet IEEE 802.3 minimum), each wrapped with a header. The frame carries: 6-byte source MAC, 6-byte destination MAC, 2-byte payload length, 2-byte sequence number, 46-byte payload, and the FCS.
3. **Codeword Generation:** Calculate and append the Checksum or CRC to the frame.
4. **Error Injection:** Inject single-bit, two-isolated, odd-numbered, burst, or targeted (word-swap / generator-multiple) errors based on the test configuration.
5. **Transmission:** Send the (corrupted or clean) codeword to the receiver over UDP; one datagram carries one frame.
6. **Validation:** The receiver recomputes the Checksum/CRC of the received dataword and compares it against the received FCS.
7. **Decision:** Accept the frame if the recomputed FCS matches (equivalently, remainder zero for CRC); otherwise reject.

Input and Output Format. **Input:** a text file (`myfile.txt`) passed via command line, along with the chosen error scheme and error-type parameters. **Output:** console logs detailing the generated frames, injected errors, the receiver’s detection status (Accept/Reject), and a final

performance summary (detection rate and total execution time). The offline driver additionally emits `results.csv`.

2 Implementation

The project is implemented in C++ and modularized into several components to separate concerns and ensure reusability.

Component Breakdown.

- `sender.cpp` & `receiver.cpp`: core simulation of the network nodes over UDP, managing framing, transmission, and validation.
- `common.cpp` & `common.h`: shared FCS module — 16-bit checksum, the generic CRC engine, frame (de)serialization, and `verifyFCS`.
- `error_injector.cpp` & `error_injector.h`: logic to mutate bits within the frame based on the requested error profile.
- `framing.cpp` & `framing.h`: encapsulation of raw data into fixed-size frames.
- `socket_util.h`: POSIX UDP networking helpers.
- `driver.cpp`: orchestrator that runs automated trials and writes `results.csv`.
- `Makefile`: automates the build.

Algorithm Implementations.

- **16-bit Checksum**: the dataword is divided into 16-bit words and added using 1's-complement arithmetic (with end-around carry). The final sum is complemented to produce the checksum.
- **CRC (generic engine)**: implemented as bit-serial polynomial long division over $\text{GF}(2)$, where subtraction is XOR. The dataword bits are appended with n zero bits ($n = \text{polynomial degree}$); the generator is XORed in wherever the leading bit is 1; the final n -bit remainder is the FCS. Figure 3 shows a small worked example. The four generator polynomials are:
 - **CRC-8**: $x^8 + x^7 + x^6 + x^4 + x^2 + 1$ (6 terms)
 - **CRC-10**: $x^{10} + x^9 + x^5 + x^4 + x + 1$ (6 terms)
 - **CRC-16**: $x^{16} + x^{15} + x^2 + 1$ (4 terms)
 - **CRC-32**: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ (15 terms)

CRC long division over GF(2) (XOR = subtraction)	
1 1 0 1 0 1 1 0 1 1 0 0 0 0	augmented dataword (1101011011 + 4 zeros)
<u>1 1 0 1 0 1 1 0 1 1 0 0 0 0</u>	pos 0: bit=1 -> XOR 10011
0 1 0 0 1 1 1 0 1 1 0 0 0 0	result
<u>0 1 0 0 1 1 1 0 1 1 0 0 0 0</u>	pos 1: bit=1 -> XOR 10011
0 0 0 0 0 0 1 0 1 1 0 0 0 0	result
<u>0 0 0 0 0 0 1 0 1 1 0 0 0 0</u>	pos 6: bit=1 -> XOR 10011
0 0 0 0 0 0 0 0 0 1 0 1 0 0 0	result
<u>0 0 0 0 0 0 0 0 0 1 0 1 0 0 0</u>	pos 8: bit=1 -> XOR 10011
0 0 0 0 0 0 0 0 0 0 0 1 1 1 0	result
0 0 0 0 0 0 0 0 0 0 0 1 1 1 0	remainder (FCS) = 1110
Codeword transmitted: 1101011011 1110	
Receiver divides the received codeword by the same generator; remainder 0 -> accept, non-zero -> reject.	

Figure 3: CRC computation by long division for a short dataword (generator $x^4 + x + 1$). Red rows mark each XOR of the generator at a leading 1; the final 4-bit remainder becomes the FCS, and the transmitted codeword is dataword followed by remainder.

3 Test Cases

To verify correctness, 5,000 automated trials were executed for each scheme against each error profile. Six error modes were used: four random and two targeted (adversarial) modes designed to expose each scheme’s theoretical blind spots.

- **TC1 — Single-Bit Error:** flip one random bit. *Checking:* every scheme should reject (non-matching FCS).
- **TC2 — Two Isolated Errors:** flip two non-adjacent bits. *Checking:* whether the two errors can cancel in the sum / division.
- **TC3 — Odd Number of Errors:** flip 3 or 5 bits. *Checking:* whether schemes with an $(x+1)$ factor catch all odd errors.
- **TC4 — Burst Error:** invert a contiguous run of bits. *Checking:* resilience to continuous corruption.
- **TC5 — Word Swap (targeted):** swap two distinct 16-bit payload words. *Checking:* the checksum’s order-independence blind spot. Expected: *Checksum misses, CRC detects.*
- **TC6 — Generator Multiple (targeted):** XOR a CRC’s own generator pattern into the dataword. *Checking:* the CRC’s algebraic blind spot (error $\equiv 0 \pmod{g(x)}$). Expected: *that CRC misses, Checksum detects.*

4 Results

Metrics were collected over 5,000 trials per (scheme, error type). Table 2 reports detection rates for all six modes; Figure 4 visualizes them.

Scheme	Error Type	Detected	Missed	Detect Rate (%)
CHECKSUM16	single-bit	5000	0	100.0
CHECKSUM16	two-isolated	4871	129	97.4
CHECKSUM16	odd-errors	4992	8	99.8
CHECKSUM16	burst	5000	0	100.0
CHECKSUM16	word_swap	0	5000	0.0
CHECKSUM16	crc_multiple	5000	0	100.0
CRC8	single-bit	5000	0	100.0
CRC8	two-isolated	4923	77	98.5
CRC8	odd-errors	5000	0	100.0
CRC8	burst	5000	0	100.0
CRC8	word_swap	4866	134	97.3
CRC8	crc_multiple	0	5000	0.0
CRC10	single-bit	5000	0	100.0
CRC10	two-isolated	5000	0	100.0
CRC10	odd-errors	5000	0	100.0
CRC10	burst	5000	0	100.0
CRC10	word_swap	4997	3	99.9
CRC10	crc_multiple	0	5000	0.0
CRC16	single-bit	5000	0	100.0
CRC16	two-isolated	5000	0	100.0
CRC16	odd-errors	5000	0	100.0
CRC16	burst	5000	0	100.0
CRC16	word_swap	5000	0	100.0
CRC16	crc_multiple	0	5000	0.0
CRC32	single-bit	5000	0	100.0
CRC32	two-isolated	5000	0	100.0
CRC32	odd-errors	5000	0	100.0
CRC32	burst	5000	0	100.0
CRC32	word_swap	5000	0	100.0
CRC32	crc_multiple	0	5000	0.0

Table 2: Detection performance across all error modes (5,000 trials each).

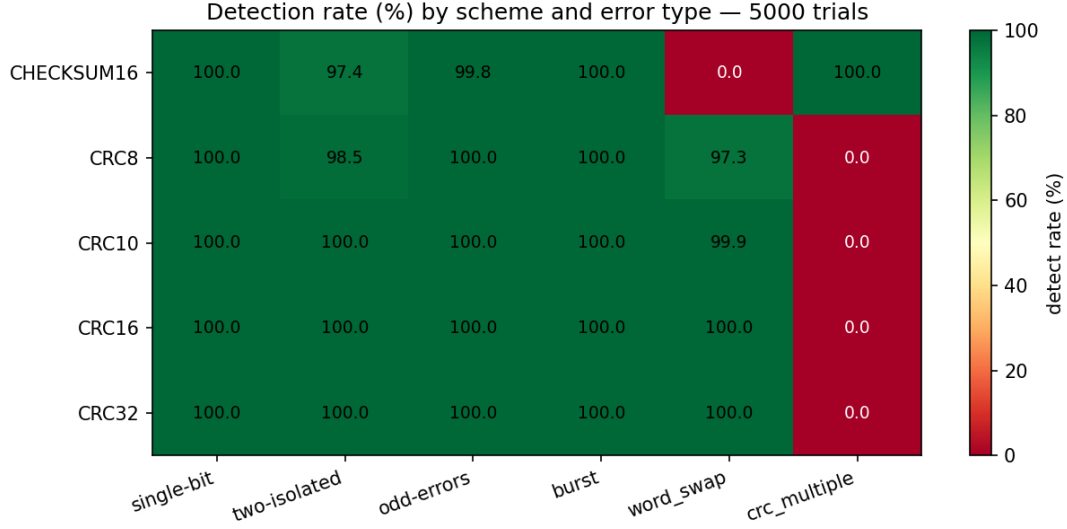


Figure 4: Detection-rate heatmap. Green = detected; red = systematic blind spot. Checksum is blind to `word_swap`; every CRC is blind to a multiple of its own generator (`crc_multiple`).

4.1 The three required comparison cases

Using a single frame checked by both CHECKSUM16 and CRC-16, the three required cases are demonstrated directly in Table 3.

Error	CHECKSUM16	CRC-16	Required case
single-bit	DETECT	DETECT	detected by both
word_swap	MISS	DETECT	detected by CRC but not Checksum
crc_multiple	DETECT	MISS	detected by Checksum but not CRC

Table 3: The three required detection-comparison cases.

4.2 Execution time and resource utilization

Timing was measured over the same trials. Figure 5 shows per-scheme FCS computation cost (mean of the non-odd modes, which isolates scheme cost); Figure 6 shows the full breakdown.

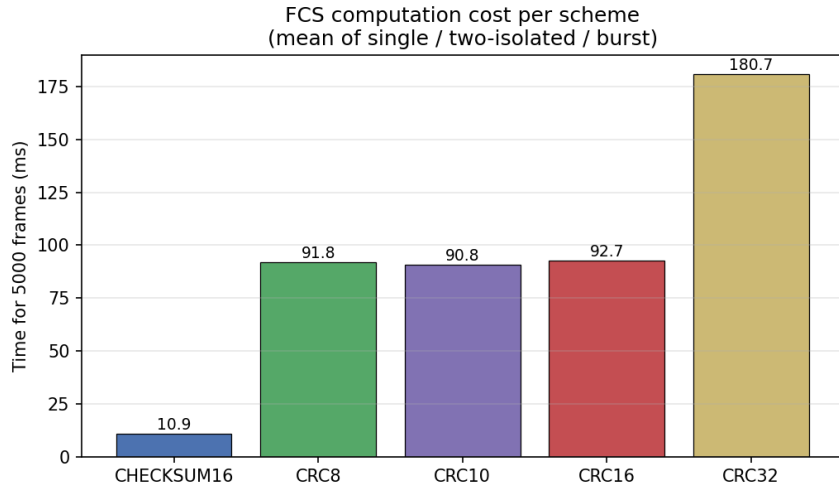


Figure 5: FCS computation cost per scheme (mean of single / two-isolated / burst).



Figure 6: Execution time by scheme and error type. The odd column is inflated by the injector’s distinct-position generation, not by the FCS algorithm.

Scheme	FCS bits	Compute (5000 frames, ms)
Checksum-16	16	~11
CRC-8	8	~92
CRC-10	10	~91
CRC-16	16	~93
CRC-32	32	~181

Table 4: Overhead (FCS width) vs. computation cost.

5 Analysis

Detection correctness (random modes). Single-bit and burst errors are caught at 100% by every scheme. Two-isolated errors expose the checksum’s first weakness: at 97.4% it misses 129/5000 cases, because two bit-flips in the *same column* of two different 16-bit words cancel under 1’s-complement addition. The CRCs are far more robust here (CRC-10/16/32 at 100%).

Targeted case 1 — word swap (checksum blind spot). Because the 1’s-complement sum is order-independent, swapping two distinct 16-bit payload words leaves the checksum unchanged, giving **0%** detection. CRC is position-sensitive and detects essentially all such swaps. (CRC-8 dips to 97.3% because its narrow 8-bit FCS occasionally aliases; wider CRCs stay at 100%.) This is the “detected by CRC but not Checksum” case.

Targeted case 2 — generator multiple (CRC blind spot). Any error pattern equal to a multiple of the generator $g(x)$ leaves the CRC remainder unchanged, so *every* CRC detects **0%** of `crc_multiple` errors built from its own generator, while the checksum — having unrelated structure — catches them at 100%. This is the “detected by Checksum but not CRC” case, and it is the fundamental reason a CRC is characterized by *which* polynomial it uses.

Odd errors and the $(x + 1)$ factor. A generator detects all odd-weight errors iff it is divisible by $(x + 1)$, which holds iff it has an *even* number of terms. Counting terms: CRC-8 (6), CRC-10 (6), CRC-16 (4) are even and therefore guarantee all-odd-error detection; **CRC-32 as specified has 15 terms — odd — so it is not divisible by $(x + 1)$ and does not carry this guarantee.** (The checksum’s 99.8% on odd errors reflects the same column-cancellation

mechanism as the two-isolated case.)

Execution time & resource utilization.

- **Checksum is cheapest by far** (~ 11 ms vs. 90–180 ms for CRC) because it sums 16-bit words in parallel, whereas the CRC engine divides bit-by-bit.
- **CRC cost is dominated by the message length**, not the degree: CRC-8/10/16 cluster around 90–93 ms because the per-bit division over the whole dataword dominates, while CRC-32 (~ 181 ms) roughly doubles the inner-loop work.
- **Odd-errors anomaly**: total time spikes (~ 442 – 554 ms) across *all* schemes for the odd mode. This is a property of the **error_injector** (its distinct-position generation loop), not of the validation algorithms — a known limitation rather than a scheme cost.
- **Overhead vs. cost**: the smallest FCS is not the cheapest to compute — checksum has a 16-bit FCS yet the lowest cost, decoupling wire overhead from CPU overhead.

Detection strength vs. cost. Figure 7 summarizes the overall trade-off. Checksum-16 sits in the cheap-but-weakest corner; CRC-32 is the strongest (widest FCS, best burst/collision resistance) but the most expensive; and CRC-8/10/16 occupy the middle. The annotation on CRC-32 marks the one counter-intuitive point: despite being the widest polynomial, it lacks an $(x + 1)$ factor and so does not guarantee detection of all odd-weight errors, unlike the narrower CRC-8/10/16.

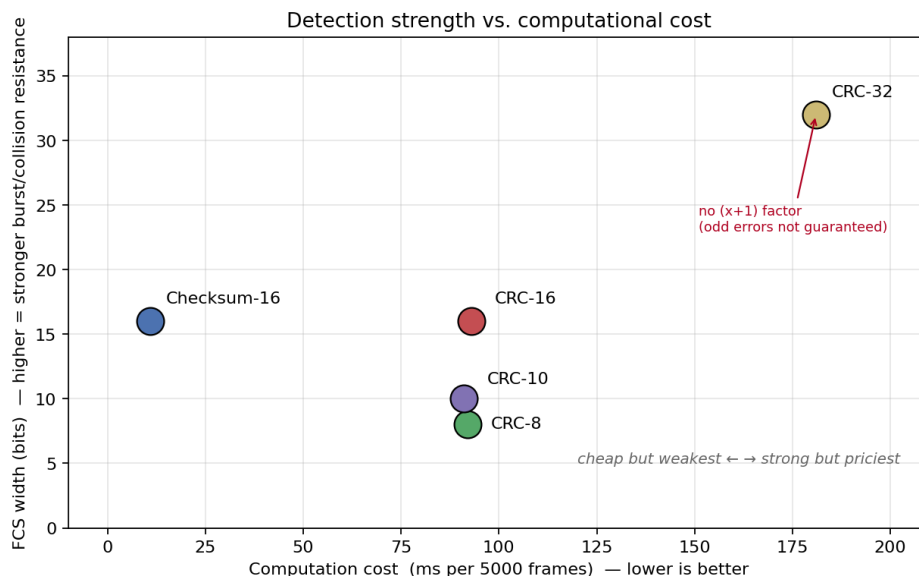


Figure 7: Detection strength (FCS width) versus computation cost. Lower-left is cheap and weak; upper-right is strong and costly.

Possible Improvements. The CRC calculation could be optimized from bit-by-bit XOR shifting to a byte-wise Look-Up Table (LUT) approach, which would substantially reduce execution time for CRC-16 and CRC-32. The odd-injector could also be rewritten to avoid its distinct-position bottleneck.

6 Comments

Evaluation of the Lab Assignment. This lab provided practical insight into the data-link-layer mechanisms that ensure data integrity. Implementing the Ethernet IEEE 802.3 framing

constraints helped bridge theory and real-world application.

Difficulty and Learning Outcomes. The assignment was moderately challenging. Translating polynomial arithmetic into bitwise C++ required careful attention to bit ordering and remainder packing. Building the modular structure with separate injectors and framing handlers reinforced good software-engineering practice. Most instructive was seeing the theoretical blind spots appear empirically: random errors alone give a misleading 100% everywhere, and only the targeted `word_swap` and `crc_multiple` cases reveal why the choice of scheme (and polynomial) matters.

Suggested Improvements. Future iterations could require students to craft “poison payload” cases that explicitly force each scheme to fail (word-swap for checksum, generator-multiples for CRC), practically demonstrating why Ethernet uses CRC-32 for frame-level detection rather than a simple checksum.